

E9: 309 Advanced Deep Learning

7-10-2020

Instructor: Sriram Ganapathy
sriramg@iisc.ac.in

Teaching Assistant : Jaswanth Reddy
jaswanthk@iisc.ac.in

Schedule - MW - 330-5pm (Microsoft Teams)
<http://leap.ee.iisc.ac.in/sriram/teaching/ADL2020/>

Recap of deep learning



Some notations

✧ $x \in \mathcal{R}^D$ - input data.

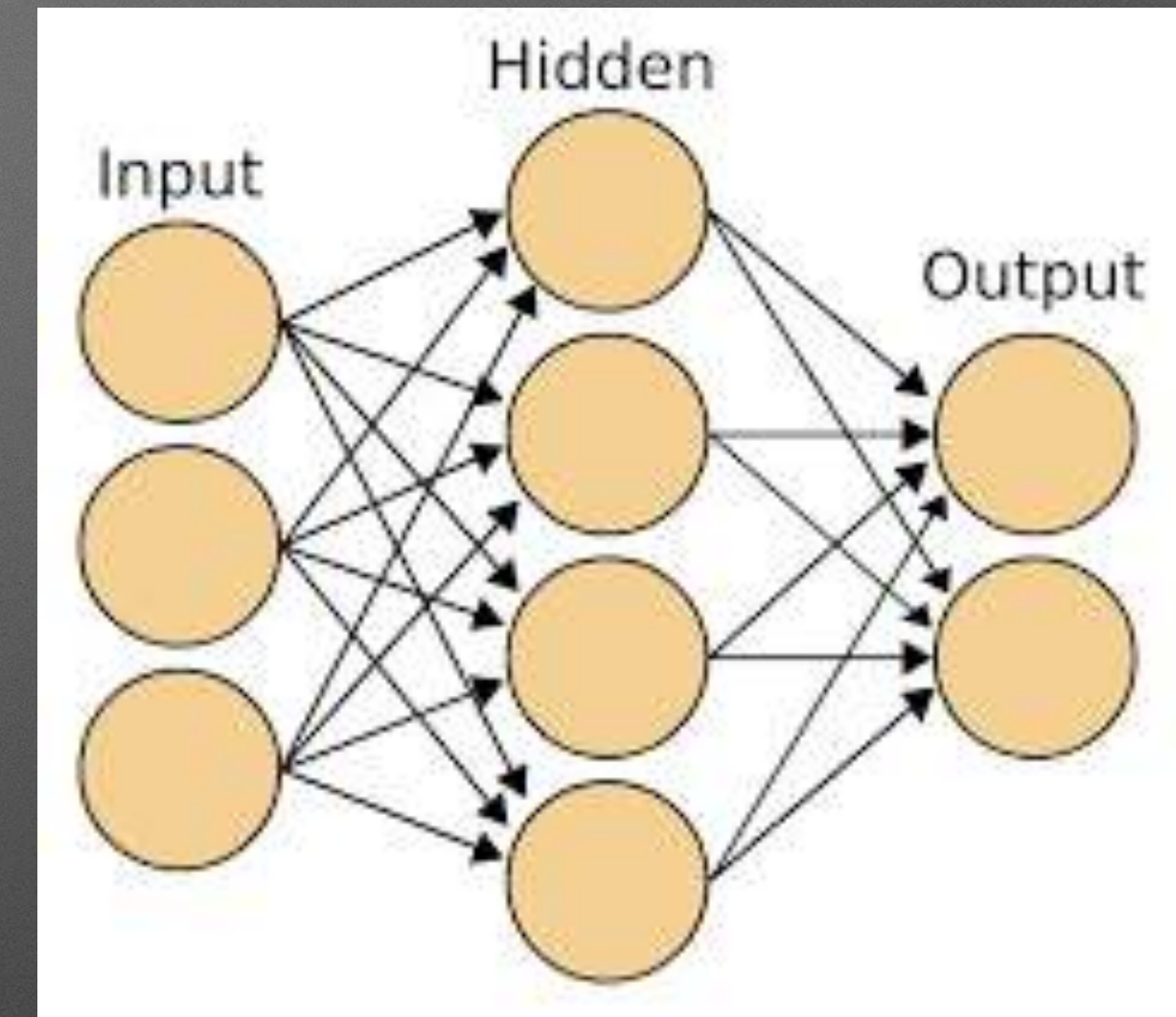
✧ $y \in \mathcal{R}^C$ - neural network targets.

✧ $\hat{y} \in \mathcal{B}^C$ - model outputs.

✧ $e, h \in \mathcal{R}^d$ - hidden model representations or embeddings.

✧ Θ - collection of learnable parameters in the model.

✧ $E(y, \hat{y})$ - error function used in the model training.



Some notations

✳ $\{x_1, \dots, x_N, y_1, \dots, y_N\}$ - labeled training data

✳ $q = \{1 \dots Q\}$ - iteration index. *epoch.*

✳ $t = \{1 \dots T\}$ - discrete time index.

✳ $l = \{1 \dots L\}$ - layer index

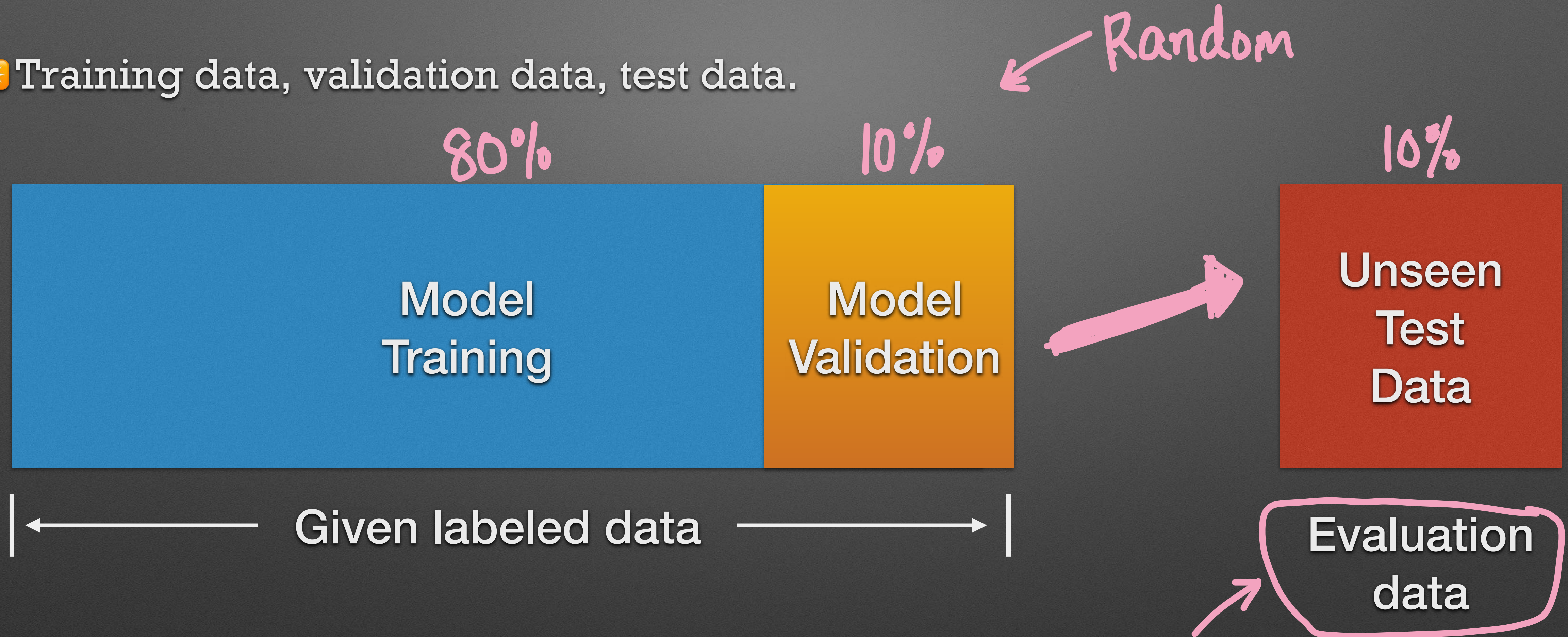
✳ η - learning rate (hyper-parameter)

✳ N_b - mini-batch size and B is the number of mini-batches.



Premise

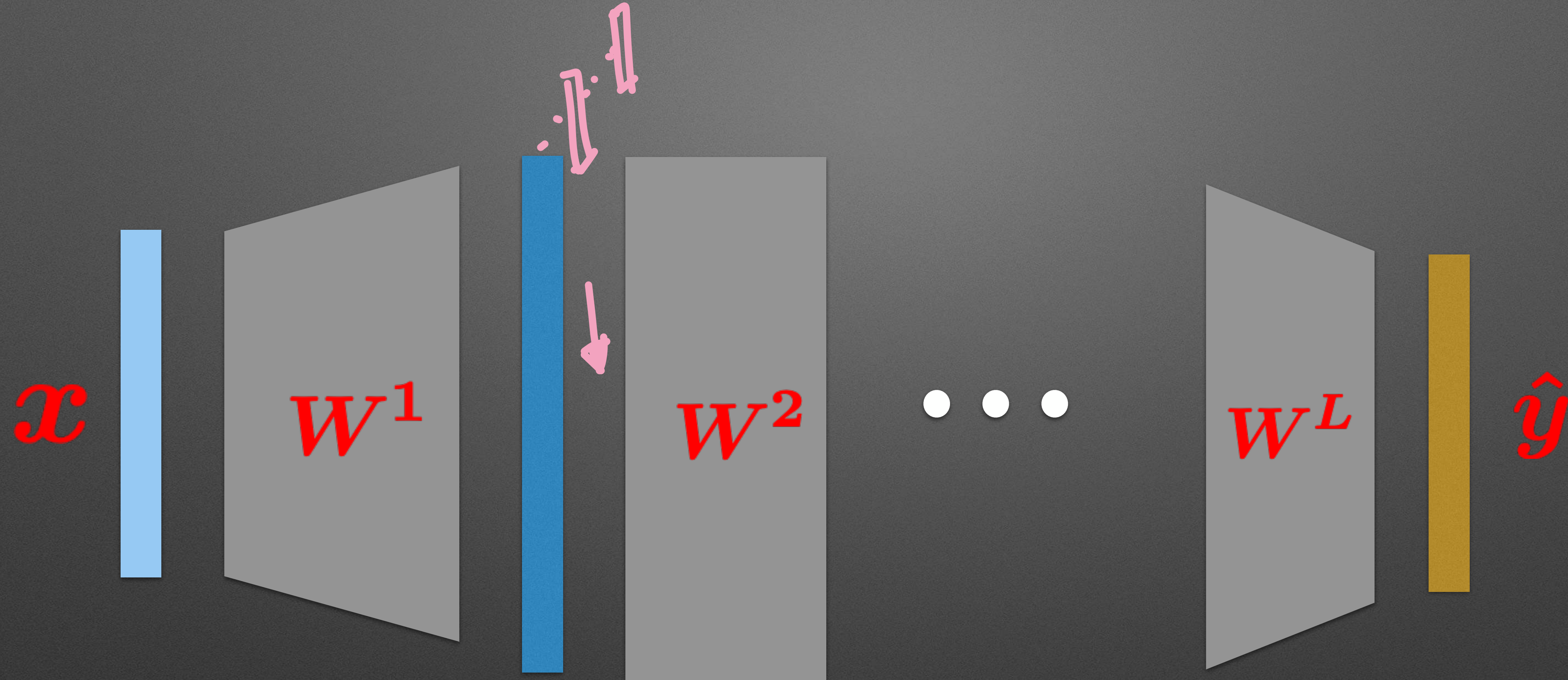
✳ Training data, validation data, test data.



✳ Model training data - used for parameter learning.

✳ Validation data - used for hyper-parameter tuning (cross validation CV).

Feedforward networks



✳ Dense connections between the input and output - also called fully connected network.

Learning in feedforward networks

$\{x_1 \dots x_N\}$ $\{y_1 \dots y_N\}$

✳ Stochastic gradient descent (SGD) - Initialize the model parameters $\Theta^{0,0}$ (randomly)

for $q = \{1 \dots Q\}$:
 for $b = \{1 \dots B\}$:

$$\Theta^{q,b} = \Theta^{q,b-1} - \eta \frac{\partial E(\{y_{b,1}, \dots, y_{b,N_b}\}, \{\hat{y}_{b,1}, \dots, \hat{y}_{b,N_b}\})}{\partial \Theta}$$

$$\Theta^{q+1,0} = \Theta^{q,B}$$

return $\Theta^* = \Theta^{Q,B}$

Batch size $\uparrow$
 $\downarrow$

Convergence
 Speed $\downarrow$
 $\uparrow$

Grad. Acc. $\uparrow$
 $\downarrow$

$$\Theta = \Theta^{q,b-1}$$



Learning in feedforward networks

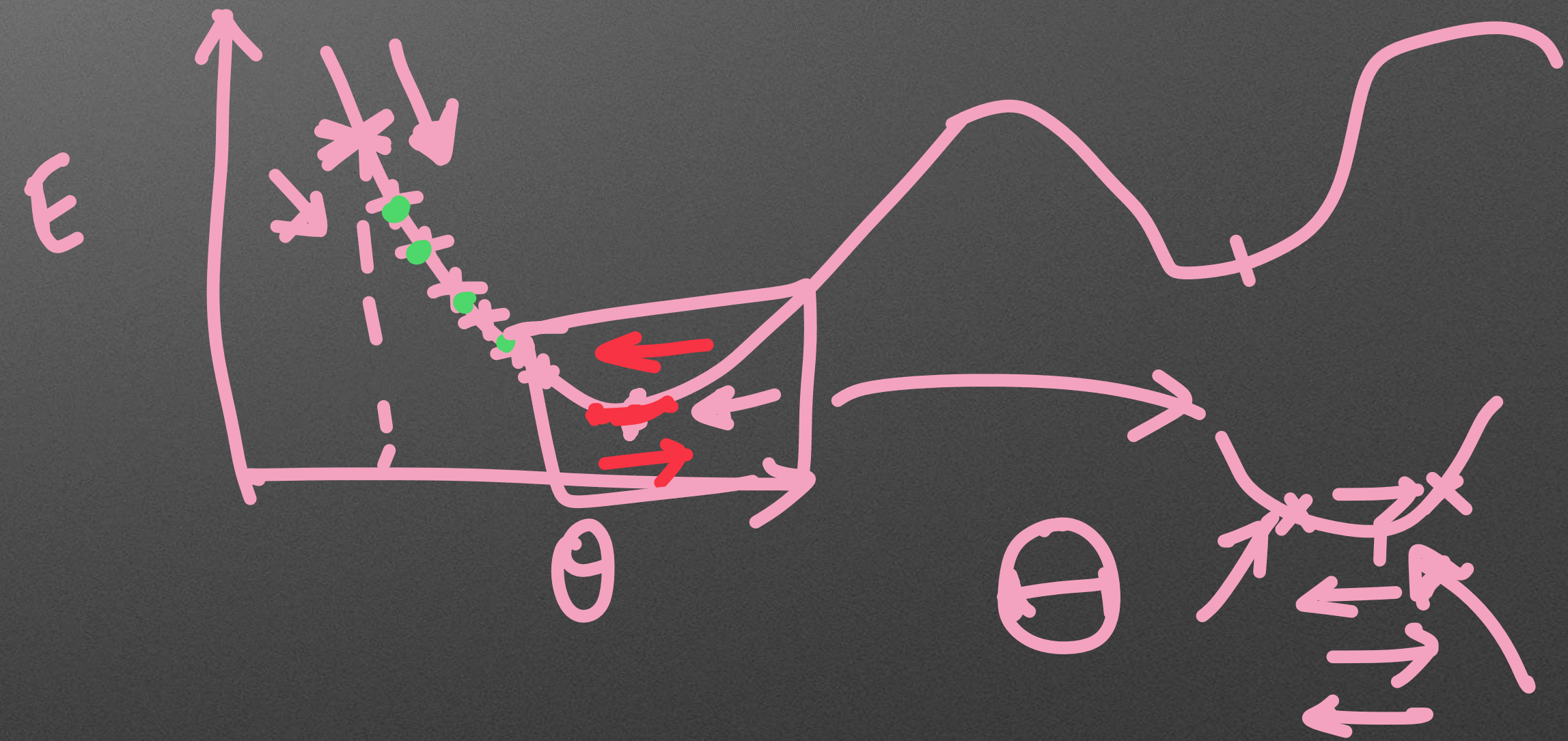
- ✳ Learning with momentum - accelerate the learning by adding a component of the previous gradient computation.

- ✳ RMSprop $\eta' = \frac{\eta}{RMS(g)}$

- ✳ Adam - adaptive moment estimation

- ➡ combines momentum and RMSprop.

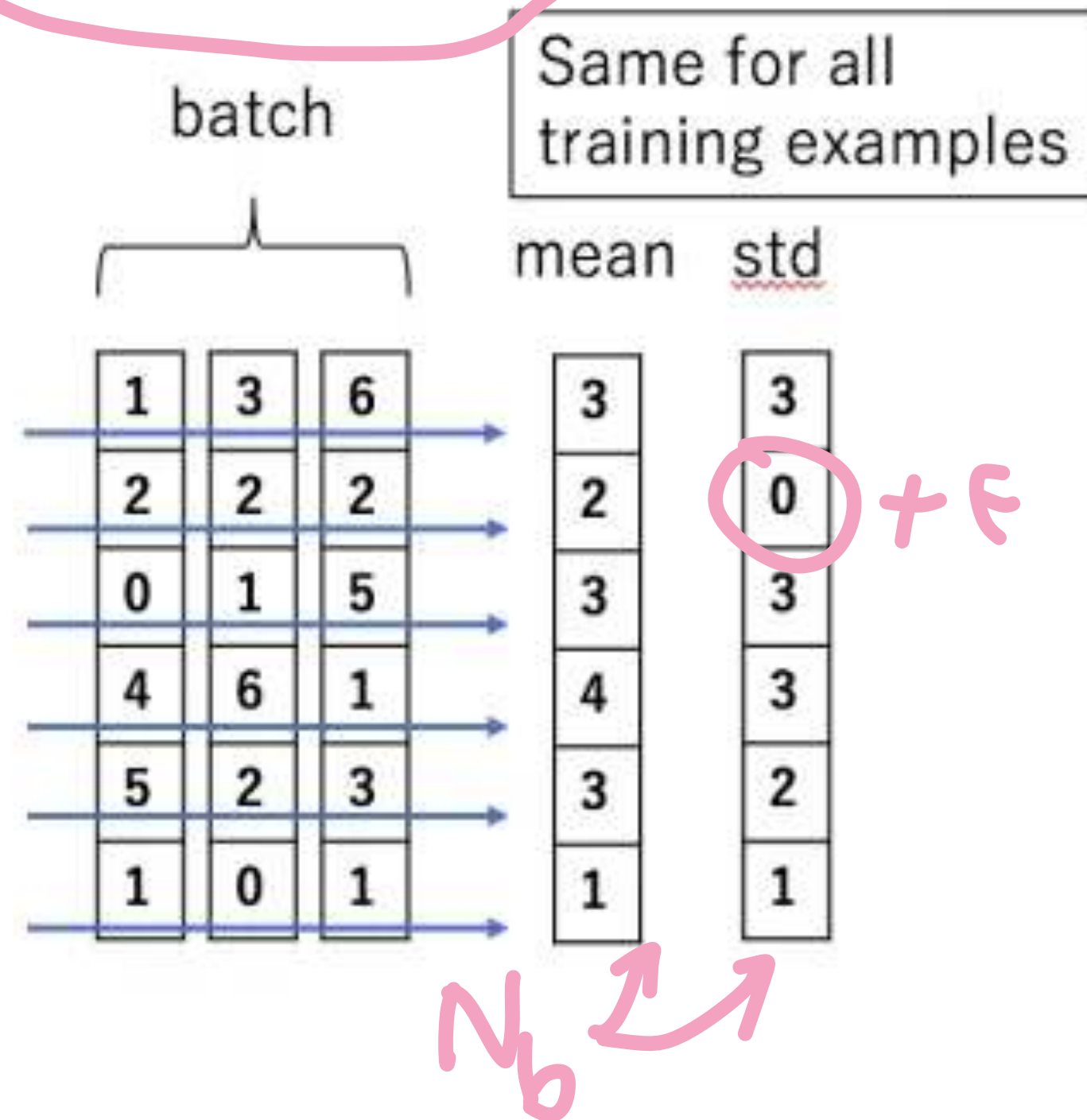
- ➡ empirically shown to be effective in many applications.



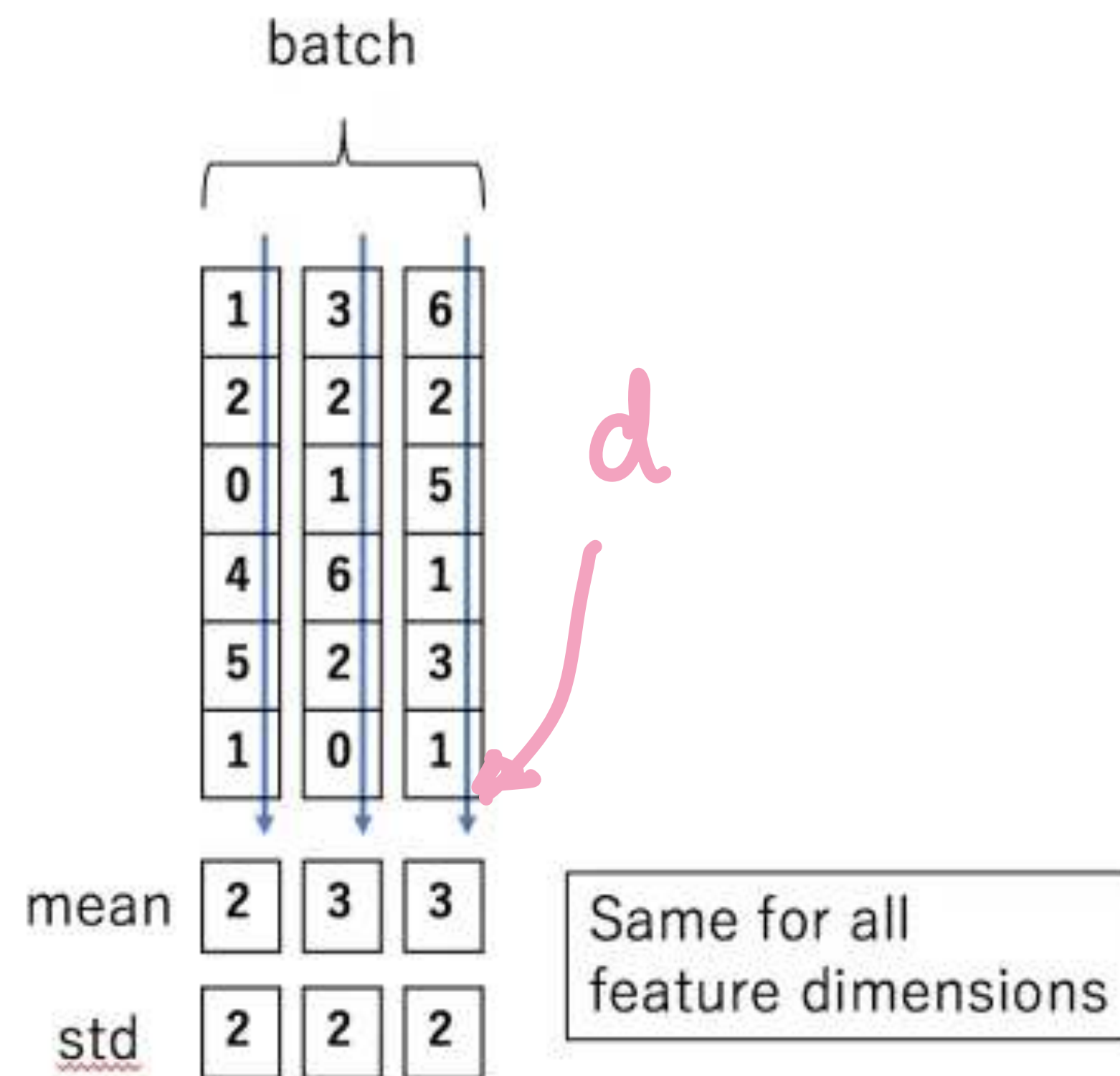
Reading assignment - Overview of gradient descent algorithms
<https://arxiv.org/pdf/1609.04747.pdf>

Normalization

Batch Normalization



Layer Normalization



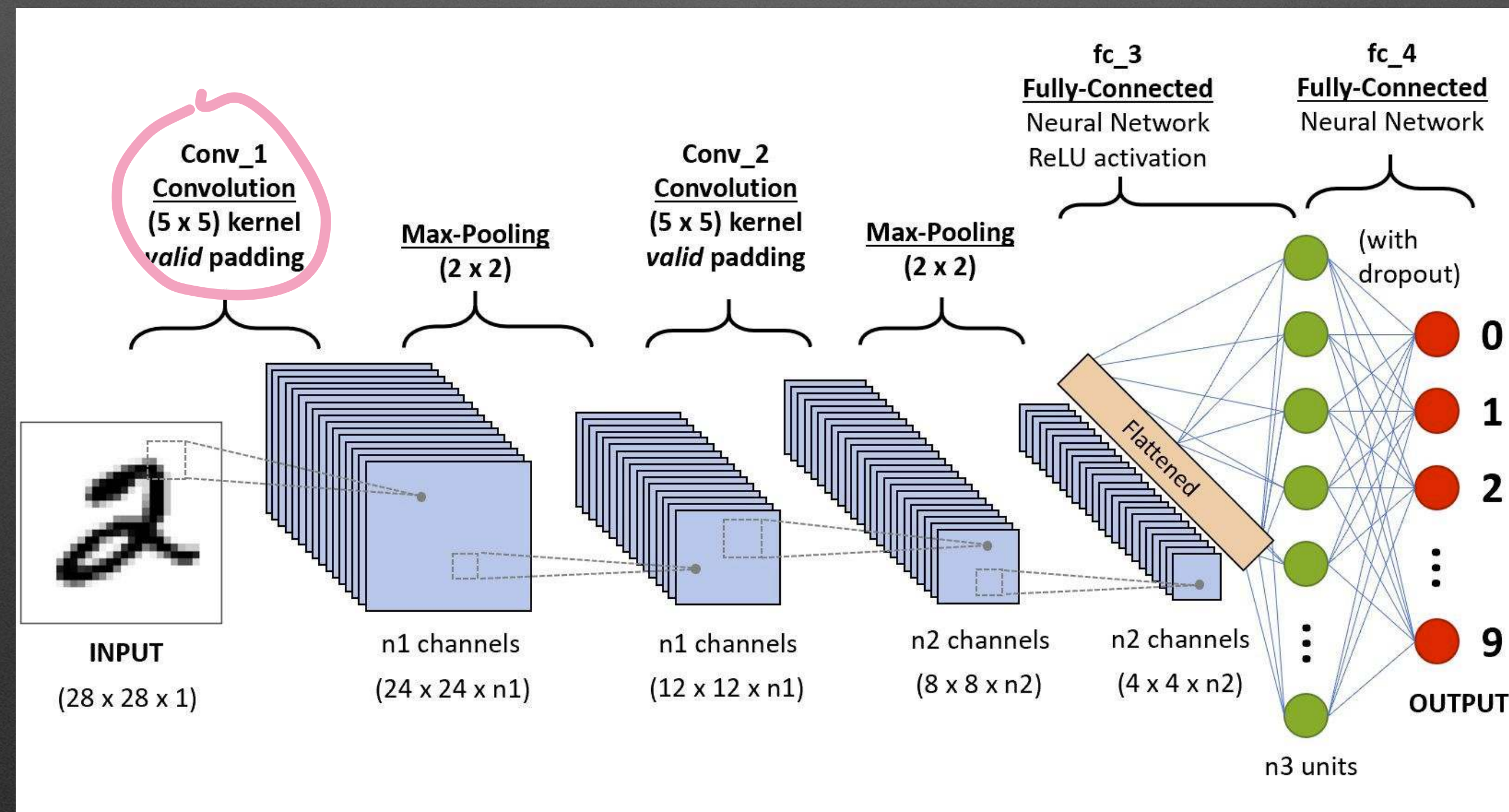
Reading assignment - How does batchnorm help optimization
<https://arxiv.org/pdf/1805.11604.pdf>

Convolutional neural networks

- ✳ Replacing affine transformations with convolutional operations

$$H(m, n) = X * W(m, n) = \sum_{i, j} X(m + i, n + j) W(i, j)$$

- ✳ Usually used with max-pooling based sub-sampling



Module - I Visual and Time Series Modeling



Why do need recurrent models

✳ Learn from ordered pairs of x, y

✓ All the data samples are treated independently.

★ Data are shuffled before mini-batch formation

✳ If the input data and output labels are time-series data $x(t), y(t)$

◉ DNNs/CNNs may fail to model the correlation of the data across the time

◉ **Question** - how can we build models that capture the time evolution of the data and the labels.



Why do need recurrent models

✳ An interesting subset of this problem is where the input alone is a time series $x(t), y$ or have different indices $x(t), y(u)$

✳ Examples

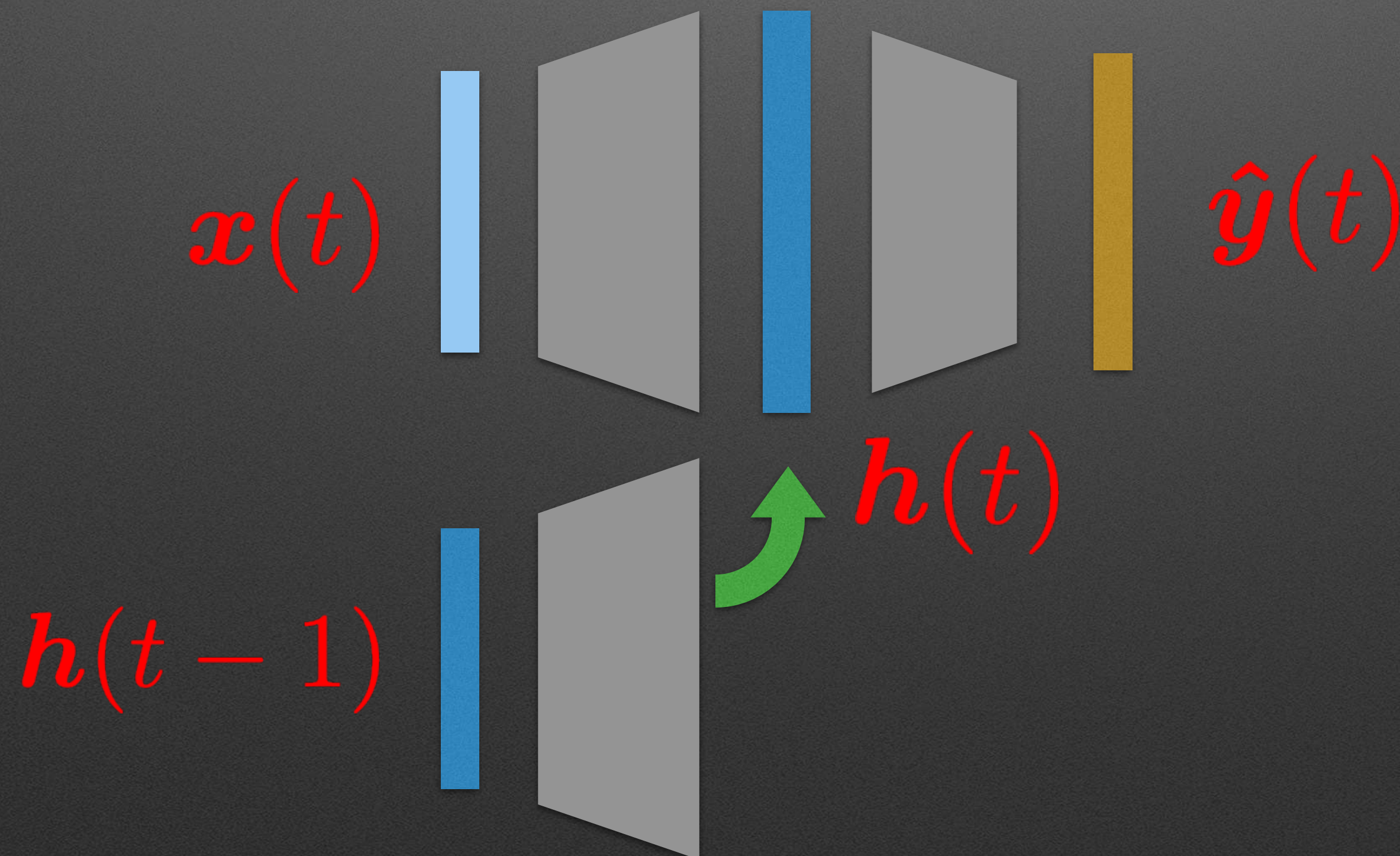
- ✓ Text sequences
- ✓ Speech and audio
- ✓ Video sequences



First order recurrence - hidden layer

- ✳ Making the hidden layer a function of the previous outputs from the hidden layer along with the input

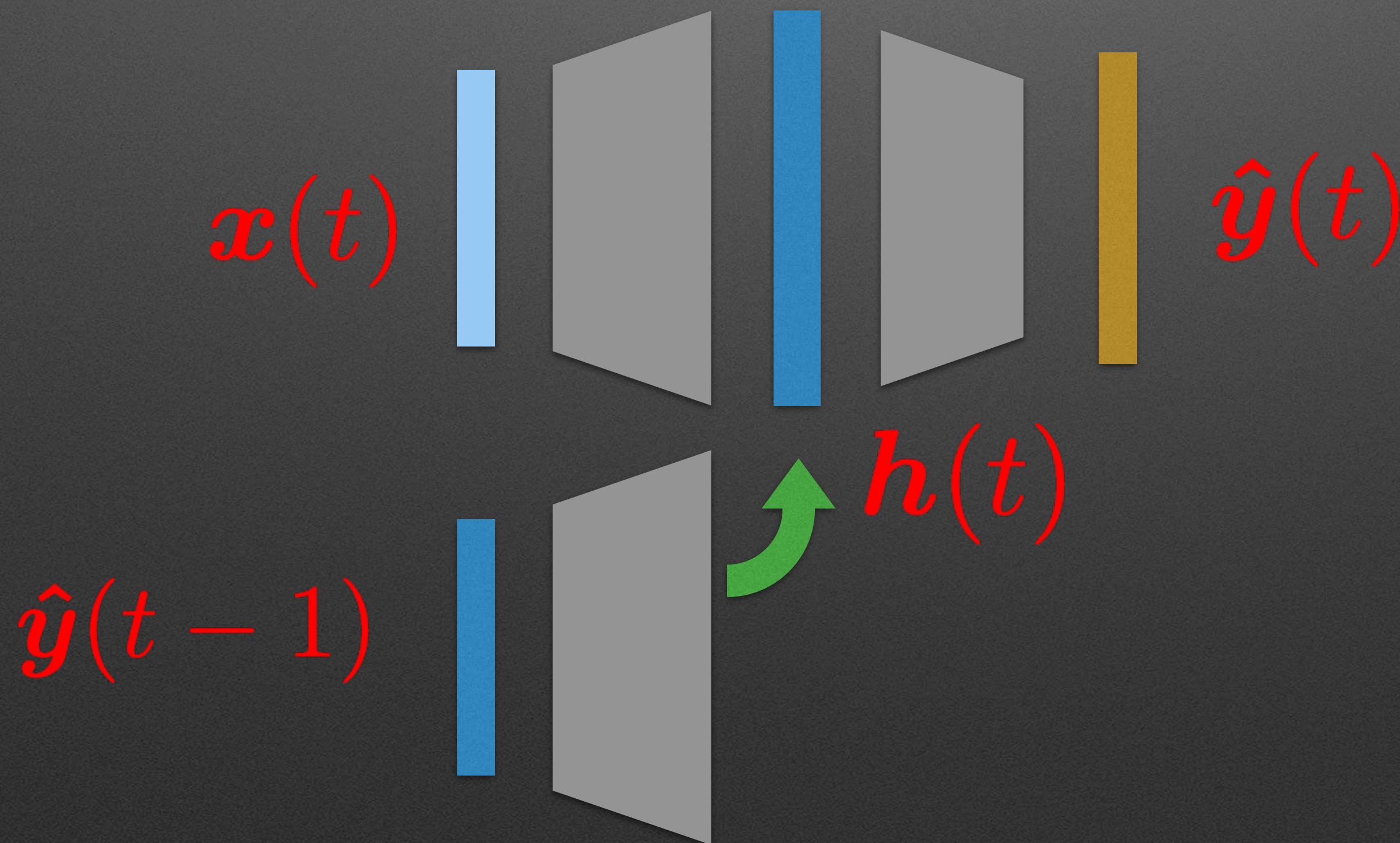
$$h(t) = f(h(t-1), x(t))$$



First order recurrence - output layer

- ✳ Making the hidden layer a function of the previous outputs from the hidden layer along with the input

$$h(t) = f(\hat{y}(t-1), x(t))$$



Looking forward (next lecture)

- ✧ Learning in recurrence networks: Back-propagation in time.
- ✧ Unsupervised Learning: Issues with forgetting and long-short-term memory networks



The bell has rung



<http://phdcomics.com/>